

第八届中国研究生人工智能创新大赛

ESASR

证据状态驱动的自适应学术检索框架

Evidence-State Adaptive Scholarly Retrieval

项目使用说明与代码仓库

比赛提交附件



github.com/wanglei1123/ScholarSeeker

团队名称: Justifying
参赛组别: 企业赛题组
文档版本: 1.0
核验日期: 2026 年 8 月 26 日

目录

1	附件用途与交付信息	1
1.1	代码仓库	1
1.2	仓库内容	1
2	运行条件与部署边界	1
2.1	必需环境	2
2.2	默认端口与服务	2
3	推荐启动方式：GitHub CLI 安装	2
3.1	安装、初始化与启动	2
3.2	环境诊断	3
3.3	免全局安装方式	3
4	备用启动方式：克隆源码并部署	3
4.1	获取代码并启动	3
4.2	配置大模型与学术数据源	3
4.3	启用本地 Cross Encoder	4
5	系统使用流程	5
5.1	完成一次复杂学术检索	5
5.2	评委应重点观察的证据	5
6	运行维护与验收	5
6.1	常用命令	5
6.2	最小验收步骤	6
6.3	代码级验证	6
7	常见问题与安全说明	6
7.1	常见问题	6
7.2	密钥、数据与可信边界	7
8	比赛现场演示建议	7

1 附件用途与交付信息

本文档用于帮助赛事评委、技术专家和试用单位在最短时间内定位证据状态驱动的自适应学术检索框架（Evidence-State Adaptive Scholarly Retrieval, ESASR）的代码、启动系统并核验核心功能。所有命令均依据提交仓库中的实际脚本整理；项目主报告负责阐述问题、方法与实验，本附件只说明软件交付、运行步骤和可复现入口。

1.1 代码仓库

表 1: 代码仓库与提交信息

项目	内容
仓库地址	github.com/wanglei1123/ScholarSeeker
Git 克隆地址	https://github.com/wanglei1123/ScholarSeeker.git
默认分支	main
核验时远端 HEAD	f5245233f5ad1e54fdebeb33e2f49df08234be3b
问题反馈	GitHub Issues
开源许可证	MIT License

截至 2026 年 8 月 26 日，仓库地址已通过远端 Git 读取验证。为保证评审复现结果与提交材料一致，正式提交时应同时保留比赛平台中的代码压缩包，并以仓库提交哈希或 Release 标签锁定评审版本。

说明：项目正式名称及界面品牌已统一为 ESASR。为保证既有安装链接和自动化脚本可继续运行，GitHub 仓库名 ScholarSeeker、CLI 命令 scholarseeker 以及 SCHOLARSEEKER_* 环境变量作为兼容性技术标识保留，不代表另一项目名称。

1.2 仓库内容

表 2: 主要目录与职责

路径	说明
<code>scholarseeker-web/</code>	Vue 3、TypeScript 与 Vite 构成的前端界面
<code>scholarseeker-api/</code>	FastAPI 检索服务、排序模块、评测工具与自动化测试
<code>docker-compose.yml</code>	Web、API、PostgreSQL、Redis 与 Neo4j 的容器编排
<code>bin/</code> 、 <code>scripts/</code>	交互式 CLI、启动、停止、状态和日志脚本
<code>docs/EVALUATION.md</code>	Gold Set、指标、预算与消融实验协议
<code>README.md</code>	项目介绍、完整命令与开发说明

2 运行条件与部署边界

2.1 必需环境

推荐使用 macOS、Windows 11 (含 Docker Desktop) 或主流 Linux 发行版。通过 CLI 从 GitHub 安装时, 需要 Node.js 20.11 或更高版本; 两种启动方式均需要正在运行的 Docker Engine 和 Docker Compose v2。项目不要求评委预先安装 Python、PostgreSQL、Redis 或 Neo4j, 这些依赖由容器统一提供。

首次构建需要联网下载容器基础镜像。

在线论文检索需要访问 OpenAlex 与 Semantic Scholar; 调用远端大模型需要相应平台的 API Key; 启用本地 Cross Encoder 时, 首次使用还需要下载模型文件。若未配置大模型 Key, 系统仍可启动, 并使用本地规则规划器执行查询。

2.2 默认端口与服务

表 3: 默认访问地址

服务	默认端口	访问地址或作用
Web	8080	http://127.0.0.1:8080
API / Swagger	8000	http://127.0.0.1:8000/docs
API 健康检查	8000	http://127.0.0.1:8000/health
PostgreSQL	5432	业务数据持久化
Redis	6379	查询计划与检索结果缓存
Neo4j Browser	7474	http://127.0.0.1:7474
Neo4j Bolt	7687	图数据库连接

端口可在项目目录的 `.env` 中修改。CLI 安装方式会在发现端口冲突时尝试选择相邻空闲端口, 并将结果保存到本地配置; 直接使用源码启动时, 应手动修改冲突端口。

3 推荐启动方式: GitHub CLI 安装

该方式适合评委快速复现。它会从 GitHub 获取项目、运行交互式配置向导并通过 Docker 启动完整系统。

3.1 安装、初始化与启动

```
npm install -g git+https://github.com/wanglei1123/ScholarSeeker.git
scholarseeker init my-scholarseeker
cd my-scholarseeker
scholarseeker start
```

初始化向导将引导用户选择 DeepSeek、Qwen、OpenAI、Kimi 或自定义兼容平台, 并以隐藏输入方式接收 API Key。向导会创建本地 `.env`、`config.yaml` 和随机 JWT 密钥; 密钥仅保存在当前计算机, 不进入前端构建产物。

说明: 若只需验证基本界面和无 Key 降级能力, 可不填写大模型 Key。此时查询规划器使用本地规则, 但在线学术数据源仍需要可访问的网络环境。

3.2 环境诊断

启动前或发生异常时，先执行：

```
scholarseeker doctor
scholarseeker config
scholarseeker status
```

doctor 检查 Docker 与配置；config 安全显示配置状态而不输出 Key 内容；status 显示各容器是否正常运行。

3.3 免全局安装方式

不希望向系统写入全局 npm 包时，可直接运行 GitHub 包：

```
npm exec --yes \
  --package=git+https://github.com/wanglei1123/ScholarSeeker.git \
  -- scholarseeker init my-scholarseeker
```

4 备用启动方式：克隆源码并部署

该方式适合需要检查代码、运行测试或修改配置的技术评委。

4.1 获取代码并启动

```
git clone https://github.com/wanglei1123/ScholarSeeker.git
cd ScholarSeeker
./run.sh
```

首次执行 ./run.sh 时，脚本会从模板创建 .env 和 config.yaml，构建 Web 与 API 镜像，启动 PostgreSQL、Redis、Neo4j、API 和 Web，并等待健康检查通过。终端最终会打印实际 Web、API 文档和 Neo4j 地址。

4.2 配置大模型与学术数据源

编辑项目根目录下的 .env。以下仅为字段示例，真实 Key 不应写入报告、截图或 Git 仓库：

```
LLM_ACTIVE_PROVIDER=deepseek
DEEPSEEK_API_KEY=
QWEN_API_KEY=
OPENAI_API_KEY=
KIMI_API_KEY=
SEMANTIC_SCHOLAR_API_KEY=
OPENALEX_API_KEY=
OPENALEX_EMAIL=
```

支持的平台标识为 deepseek、qwen、openai、kimi 和 custom。

OpenAlex 与 Semantic Scholar 的 Key 为可选项，但配置后通常可获得更稳定的访问额度。修改配置后，使用源码方式或 CLI 的无构建重启选项使新配置生效。

4.3 启用本地 **Cross Encoder**

默认镜像不安装体积较大的重排依赖。如需复现局部语义精排与自适应结果集，使用：

```
./run.sh --with-reranker
```

该命令会以 `INSTALL_RERANKER=true` 和 `CROSS_ENCODER_ENABLED=true` 构建 API 镜像。首次构建及首次模型下载耗时明显更长；若模型不可用，系统会回退到基础融合排序并记录降级原因，不会伪造精排结果。

5 系统使用流程

5.1 完成一次复杂学术检索

1. 打开 Web 地址并进入智能检索页面。
2. 输入包含研究主题、方法、年份、数据集、Venue、开放获取或排除条件的自然语言问题。
3. 提交查询，观察系统生成的结构化约束、子查询和检索阶段状态。
4. 在结果页检查论文标题、作者、年份、来源、推荐理由、约束命中情况和缺失元数据。
5. 进入论文详情页核验摘要、DOI、开放获取链接及关系图；关系类型会区分真实引用与算法相关。
6. 根据需要收藏、管理或导出论文记录；导出链接优先使用 DOI，其次使用数据源提供的原始 URL。

可用于现场演示的复合查询示例：

查找 2022 年以后使用多模态大模型进行医学影像诊断、公开代码并发表于高水平会议或期刊的论文，排除纯文本医疗问答工作。

5.2 评委应重点观察的证据

- 查询中的主题、方法、年份、Venue、开放获取和排除条件是否被结构化保留；
- 首轮证据不足时，系统是否说明缺口并在预算允许时补检；
- 多源结果是否按 DOI、来源 ID 或规范化标题去重；
- 结果是否提供来源、命中约束、缺失字段、排序阶段和成本轨迹；
- 数据源或模型故障时，界面是否明确显示降级，而不是以演示数据替代在线结果。

6 运行维护与验收

6.1 常用命令

命令	作用
<code>scholarseeker status</code>	查看容器状态
<code>scholarseeker logs api web</code>	持续查看 API 和 Web 日志
<code>scholarseeker restart --no-build</code>	配置修改后快速重启
<code>scholarseeker provider list</code>	查看平台、Key 状态和默认平台
<code>scholarseeker provider use qwen</code>	切换默认大模型平台
<code>scholarseeker stop</code>	停止服务并保留数据卷

命令	作用
<code>./scripts/status.sh</code>	源码方式查看服务状态
<code>./scripts/logs.sh api web</code>	源码方式查看指定日志
<code>./scripts/stop.sh</code>	停止服务并保留数据
<code>./scripts/stop.sh --volumes</code>	删除容器及全部本地数据卷

说明: `--volumes` 会删除 PostgreSQL、Redis、Neo4j 数据和模型缓存, 仅应在确认本地数据可丢弃时使用。

6.2 最小验收步骤

1. 执行 `scholarseeker doctor` 或确认 Docker Compose v2 正常。
2. 启动后执行 `docker compose ps`, 确认五个服务处于运行或健康状态。
3. 访问 <http://127.0.0.1:8000/health>; 依赖正常时返回健康状态, 依赖异常时返回 HTTP 503 和 `overall=degraded`。
4. 访问 Web 首页, 完成至少一次复合查询, 并在详情页核验来源与缺失字段。
5. 停止服务后重新启动, 确认数据库与收藏等持久化数据仍然存在。

6.3 代码级验证

若在源码目录中进行完整技术验收, 可执行:

```
docker compose exec api python -m unittest discover -s tests -v
cd scholarseeker-web && npm ci && npm run build
cd .. && npm test && npm run pack:check
```

第一条命令在已启动的 API 容器内运行后端测试; 第二组命令完成前端类型检查和生产构建; 第三组命令验证 CLI 行为及 npm 发布包内容。离线检索实验与指标复现的完整数据格式和命令见仓库中的 `docs/EVALUATION.md`。

7 常见问题与安全说明

7.1 常见问题

现象	处理方法
提示 Docker 未运行	启动 Docker Desktop 或 Docker Engine, 再执行 <code>scholarseeker doctor</code> 。
端口已被占用	CLI 会尝试相邻端口; 源码方式请修改 <code>.env</code> 中的端口字段。
API 长时间未就绪	查看 <code>scholarseeker logs api postgres</code> 或 <code>./scripts/logs.sh api postgres</code> 。

现象	处理方法
PostgreSQL 密码认证失败	旧数据卷可能使用了不同密码。优先恢复原密码；只有旧数据可丢弃时才执行 <code>./scripts/stop.sh --volumes</code> 。
未配置 LLM Key	系统使用本地规则规划；这属于显式降级，不影响服务启动。
Cross Encoder 首次启动较慢	等待模型下载并检查 API 日志；无法加载时系统自动退回基础排序。
健康检查返回 degraded	按返回字段定位 PostgreSQL、Redis 或 Neo4j 异常，并检查对应容器日志。
在线结果数量波动	OpenAlex 与 Semantic Scholar 响应可能随时间变化；正式效果比较应使用冻结候选或仓库中的离线评测协议。

7.2 密钥、数据与可信边界

- 不得把真实 API Key、JWT 密钥或数据库密码提交到 GitHub；`.env` 和 `config.yaml` 仅保存在本地。
- 在线主检索链路以 OpenAlex 与 Semantic Scholar 为主要来源。数据源失败、字段缺失和模型降级会进入检索轨迹，不应被解释为完整在线结果。
- 论文标题、作者、年份、DOI 与开放获取状态来自外部数据源，使用者在正式引用前应通过 DOI 或出版社页面再次核验。
- 系统用于辅助文献发现与证据组织，不替代研究者的学术判断、系统综述筛选流程或版权合规审查。

8 比赛现场演示建议

建议按“环境可信、查询复杂、证据可查、故障可见”的顺序完成五分钟演示：

1. **30 秒**：展示本附件首页二维码、GitHub 仓库和 `scholarseeker doctor` 结果；
2. **60 秒**：输入包含多重约束的真实科研问题，展示结构化查询计划；
3. **90 秒**：展示多源召回、去重、约束覆盖、缺口补检和排序轨迹；
4. **90 秒**：打开论文详情，核验 DOI、开放获取、缺失字段和关系类型；
5. **30 秒**：展示 API 健康检查、成本指标和无 Key 或模型异常时的显式降级。

正式提交前应完成以下检查：仓库对评委可访问；提交哈希或 Release 标签与代码压缩包一致；README 与本附件中的命令可执行；仓库不含任何真实密钥；演示机器已预拉取容器镜像和可选重排模型；准备一份冻结查询结果用于网络波动时解释系统边界，但不得冒充实时在线结果。